OPERATOR GUIDE | SOFTWARE PIPELINE | API REFERENCE

AI-Powered Urban Digital Twin

A detailed manual for using the PBL prototype and understanding how its map, backend, simulation, and scenario tools work.

SEARCH A PLACE
Find a real place and inspect mapped roads.

PLAY A SCENARIO
Animate a local road model and compare what-if estimates.

Prototype status

The searched road geography comes from OpenStreetMap. Vehicle movement, congestion labels, dashboard readings, forecasts, and scenario outcomes are prototype estimates. This manual distinguishes connected map data from illustrative outputs.

01 / PROJECT MANUAL

Start and open the app

The development launcher starts the local API and Vite frontend together. A browser address printed by Vite is the app entry point.

Requirements

Install Node.js 20 or newer. Open PowerShell in the PBL project folder. The first run downloads npm packages declared in **package.json**.

```
npm install
npm run dev
```

When Vite reports a Local URL, open it in your browser. The API listens on **http://127.0.0.1:8000**. Keep the terminal open while using the app. Stop processes started by the launcher with Ctrl+C.

If you see EADDRINUSE on port 8000

The launcher checks `/api/health` and reuses an existing Urban Twin API. If another application owns port 8000, stop that process or change the API port and matching Vite proxy together.

First screen tour

Area	What it contains
Left navigation	Digital twin, live operations, incidents, scenario studio, outcome comparison, and settings.
Map workspace	Illustrated demo city, search, zoom, 2D/3D view, four map layers, and sample incident markers. Time-window tabs are display controls; they do not fetch history.
Layer toggles	Traffic shows sample colored overlays; Incidents shows sample markers; Weather shows a sample rain area; Infrastructure shows sample asset pins.
Insight column	Demo traffic outlook, anomaly cards, decision-support text, and scenario entry points.
Theme control	Cycles through dark, light, and system appearance. The preference is saved in browser local storage.

Demo data label

The default dashboard is an illustrative city view with sample readings and sample incidents. Searching a place switches to an OpenStreetMap base map and road model for the selected area.

02 / PROJECT MANUAL

Search a place and build its road model

The location workflow uses geocoding to find coordinates, then requests road geometry for a bounded area around the selected result.

Step	Action	What happens
1	Search a city, place, or postcode. Press Enter or the arrow button.	The backend asks Open-Meteo for up to five matching place results.
2	Choose the right result from the list.	The map centers on its latitude and longitude. The location screen shows OpenStreetMap tiles.
3	Set Area radius and choose Create local simulation model.	Radius can be 400 to 1,500 metres. Default is 800 metres. The backend requests mapped roads in that area.
4	Wait for Road model ready.	The card reports road segment count, approximate intersections, and total road length in kilometres. A paused simulation is created.
5	Click a road line.	A popup shows the mapped road name, highway class, lane count, and one-way tag when available.

What the model uses

The selected point and radius define a north/south/east/west bounding box. The Overpass query requests mapped highway ways and their geometry. The backend keeps each road segment ID, coordinates, name or reference, road class, one-way tag, lane count, and calculated length. It uses a geodesic distance calculation for segment lengths.

The simplified road filter includes motorway, trunk, primary, secondary, tertiary, residential, service, and related mapped road classes. The model does not include traffic sensor feeds, traffic signals, building geometry, public transport, emergency fleet locations, or complete lane-level routing. Some map attributes may be missing for a particular road.

Map attribution

Keep the visible OpenStreetMap contributor attribution in the app. The map tiles and road geometry depend on network access and public provider availability.

03 / PROJECT MANUAL

Play controls and read the map

Once the local road model is ready, the map animates sample vehicles and recolors each road segment according to its generated traffic state.

Control or mark	Meaning
Play / Pause	Starts or stops periodic simulation ticks. The frontend requests an updated snapshot about every 900 milliseconds.
Reset	Pauses and returns the step counter to zero. The speed selection returns to 1x.
Speed	Cycles through 1x, 2x, and 4x. It changes how far vehicle markers move per tick.
T+ minutes	The displayed clock advances by 5 simulation minutes per tick.
Green road	Free: generated road-state value below 42.
Amber road	Slow: generated value from 42 through 71.
Red road	Heavy: generated value 72 or higher.
Blue / orange-red dots	Synthetic vehicle markers. Most are blue; every fifth marker uses orange-red.
Green center marker / dashed circle	Selected place center and the chosen model radius.

How the moving simulation is generated

Vehicle markers are placed on selected road geometries and move to new positions using a repeating progress value. The model creates between 12 and 45 markers depending on total road length. It does not route vehicles through intersections, obey one-way directions, or match observed cars.

Each road status is generated from its map ID, position in the road list, and current simulation step. The status changes with the step. The congestion index and average speed are also generated by compact formulas. These values demonstrate a changing interface; they do not describe actual traffic conditions.

```
roadLevel = (charSum(roadId) + roadIndex * 13 + step * 7) % 100
congestionIndex = round(46 + sin(step / 4) * 13 + (vehicles / 45) * 8)
averageSpeedKph = max(12, round(52 - congestionIndex * 0.32))
```

Reading line colors

Green = free flow. Amber = slow. Red = heavy. The location view keeps these meanings even though its map legend is hidden. Road colors are calculated demo states, not measurements.

04 / PROJECT MANUAL

Use the dashboard and scenario studio

The dashboard demonstrates monitoring and decision support. Scenario controls produce transparent rule-based estimates that can be compared before a human review.

Dashboard cards

Card	What it represents in the prototype
Network speed	Sample network average speed; the default display is a fixed demo reading.
Congestion index	Sample score on a 0 to 100 scale; the default display is fixed demo data.
Weather impact	Sample rainfall rate and impact label. There is no weather API connected.
Active incidents	Sample incident count and severity. Incidents are defined in local project data.
Average travel time	Sample journey time for a demo route, not a route from the searched location.
Traffic outlook	A chart and confidence label built from local demo values; not an ML forecast.
Risk and recommendations	Illustrative alerts and decision-support text; no specialist AI agents are running.

Build a what-if scenario

Choose Configure scenario or Run simulation. Use the road-closure toggle, rainfall slider (0-100 mm/h), and demand slider (0-100% change from baseline). The prompt helper recognizes a limited set of words: open; close, closure, block, or shut; a number followed by mm or rain; peak or surge; low traffic or reduced demand. It uses keyword rules, not a language model.

Estimates are calculated locally. A penalty is formed from closure, rounded rainfall, and rounded demand. The penalty adjusts sample travel time, congestion, traffic flow, and emergency response time. The preview displays an uncertainty range as a simple percentage of travel time.

```
penalty = (roadClosed ? 8 : 0) + round(rain / 18) + round(demand / 28)
travelMin = 24 + penalty
congestion = min(99, 61 + round(penalty * 1.4))
flow = max(30, 77 - round(penalty * 1.25))
responseMin = 8 + round(penalty * 0.55)
uncertaintyMin = max(2, round(travelMin * 0.12))
```

Default controls	Illustrative result
Road closed; rain 72 mm/h; demand +58%	Travel time 38 min; congestion 81/100; flow 59%; response time 16 min; uncertainty about +/-5 min.

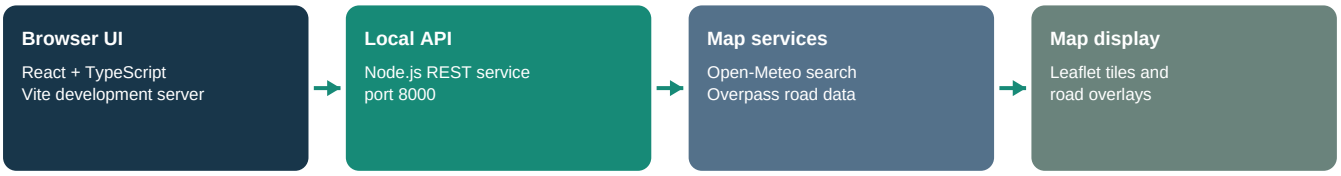
The Optimize control opens the road-closure toggle and displays a drawn route in the demo map. It is not a route-finding algorithm. Mark reviewed changes the local review state only; it does not send a command to city systems.

05 / PROJECT MANUAL

Software architecture and pipelines

This page describes the software present in the PBL folder. The original presentation also lists future or proposed technologies; those are separated from the current implementation.

Runtime request path



Road tiles load directly in the browser from OpenStreetMap. Location search and road-model requests travel through the Node API. Vite forwards browser `/api` requests to port 8000 during local development.

Installed software and embedded libraries

Component	Current role
React 18 + React DOM	Build and render the browser interface.
TypeScript 5.6	Types for UI state, roads, locations, and simulation snapshots.
Vite 6 + React plugin	Local dev server, frontend build, and /api proxy.
Leaflet 1.9 + React Leaflet 4	Map container, raster tiles, road polylines, popups, radius, and vehicle markers.
Lucide React	Interface icon set.
Custom CSS	Layout, map illustration, responsive UI, and dark/light/system themes.
Node.js built-in modules	HTTP API, fetch to external services, JSON cache files, UUIDs, and request handling.
npm	Dependency installation and run/build scripts.

Persistence

Road models are saved under `backend/data/models.json` and loaded when the API starts. Geocoding results are cached in memory for up to six hours. Active simulation state is held in memory, so a backend restart ends active simulations; a saved road model can be loaded again by rebuilding it.

Presentation technology list vs current code

The deck lists React, TypeScript, Tailwind, MapLibre/Leaflet; Python, FastAPI, REST, WebSockets; PostgreSQL, PostGIS, Redis; Python, Pandas, NumPy, Scikit-learn, PyTorch; SUMO, TraCI; and Git, GitHub, Docker. Current code uses React, TypeScript, custom CSS, Leaflet, Node.js HTTP/REST, JSON plus memory, Vite, and npm. Tailwind, MapLibre, FastAPI, WebSockets, SQL/Redis, Python ML libraries, SUMO, and TraCI are not part of this prototype. Git/GitHub/Docker are not required to launch it locally.

06 / PROJECT MANUAL

API and external integration reference

The prototype uses three external map/location services and a small local JSON API. There are no API keys or login flow in the current source.

External services

Service	Request from this project	Purpose / returned information
Open-Meteo Geocoding API	GET https://geocoding-api.open-meteo.com/v1/search Query: name, count=5, language=en, format=json	Returns place candidates, IDs, coordinates, country, and administrative region. Results are cached in memory for six hours.
OpenStreetMap Overpass API	POST https://overpass-api.de/api/interpreter Form body: data=<Overpass QL>	Returns mapped drivable road ways and geometry in the requested bounding box. Override default URL with OVERPASS_URL.
OpenStreetMap tile service	Browser requests https://tile.openstreetmap.org/{z}/{x}/{y}.png	Renders base map below the road model. The app displays OpenStreetMap contributor attribution.

Local API routes

Method and route	Input	Response / use
GET /api/health	None	Service status and count of cached models.
GET /api/geocode?q=...	Search term, 2-120 characters	Up to five place results from the geocoder.
POST /api/models	JSON: {"location":{"latitude":...,"longitude":...},"radiusMeters":800}	Road geometry, model summary, and model ID. Radius is clamped to 400-1500 metres.
POST /api/simulations	JSON: {"modelId":"..."}	Creates a paused simulation and returns its first snapshot.
GET /api/simulations/{id}/tick	Simulation ID in URL	Advances one step when playing and returns a refreshed snapshot.
PATCH /api/simulations/{id}/command	JSON action play, pause, reset, or speed. Speed value is 1, 2, or 4.	Applies playback command and returns a snapshot.
GET /api/simulations/{id}	Simulation ID in URL	Returns current simulation snapshot.

The Vite proxy maps `/api` to `http://127.0.0.1:8000`. The backend uses port 8000 by default; `API_PORT` can override it, and the Vite proxy must be changed to match. The frontend polls the tick route while playback is active. The API sends JSON and supports GET, POST, PATCH, and OPTIONS. The implementation uses REST-style polling; no WebSocket endpoint is present.

07 / PROJECT MANUAL

Boundaries, troubleshooting, and glossary

Use the prototype to demonstrate the workflow and interface. Treat outputs as estimates until connected to operational datasets and validated models.

Current scope and data boundaries

Area	Current implementation
Real map	Search uses Open-Meteo geocoding; searched-place base tiles and roads use OpenStreetMap services.
Live feeds	No live traffic sensors, weather feed, GIS database, incident management system, or emergency fleet API is connected. Dashboard values and source-health claims are demo values.
Prediction and AI	No trained ML model or AI agent is invoked. Trends, recommendations, congestion, and risk values are local demonstration logic or fixed content.
Traffic movement	Markers are synthetic and rule-driven. No calibrated demand model, routing engine, traffic signal model, or SUMO simulation is running.
Decision control	The app displays possible outcomes and review state. It does not issue road-control, routing, dispatch, or emergency-service commands.
Security / scale	This is a local student prototype. It has no user authentication, multi-user session management, production access controls, or service-level guarantees.

Troubleshooting

Symptom	Try this
Search has no results	Check spelling and add city/country. Search needs at least two characters and an internet connection.
Road model request fails	Retry later because the public Overpass service can be busy. Try a smaller radius.
Map tiles are blank	Check internet access and availability of the public tile service.
API cannot start on port 8000	The launcher reuses Urban Twin API if found. Stop another app using the port; if changing the port, update the Vite proxy too.
Simulation says connection lost	Check the terminal for the API process and reload after restarting npm run dev.

Glossary

Road segment: a mapped road geometry returned as a way. **Bounding box:** rectangular query area surrounding the selected center. **Tick:** one requested simulation update. **Snapshot:** current playback state, vehicle coordinates, road colors, and metrics. **REST API:** browser/backend request-response interface over HTTP. **Digital twin:** digital representation of a real place; this prototype models mapped road geometry for searched locations. **2D/3D toggle:** visual perspective transform on the illustrated demo map, not a 3D GIS city model.

Primary implementation files: `src/App.tsx`, `src/style.css`, `backend/server.mjs`, `scripts/dev.mjs`, `vite.config.ts`, and `package.json`. The original project scope is captured in `docs/presentation-extract.md`.